LINQ (Language Integrated Query) – Documentation

1. What is LINQ?

LINQ (Language Integrated Query) is a feature in C# that allows developers to query and manipulate data using C# syntax.

LINQ can be used with:

- Collections
- Arrays
- Lists
- Dictionaries
- Databases
- XML
- Entity Framework
- Other data sources

Without LINQ

```
List<int> numbers = new List<int>
{
    10, 20, 30, 40, 50
};

List<int> result = new List<int>();

foreach (int number in numbers)
{
    if (number > 25)
    {
        result.Add(number);
    }
}
```

Using LINQ

```
List<int> numbers = new List<int>
{
    10, 20, 30, 40, 50
};
```

```
var result = numbers.Where(x => x > 25).ToList();
```

LINQ makes the code **shorter, readable, and easier to maintain**.

---

2. Why Use LINQ?

LINQ provides several benefits:

- Easy-to-read syntax
- Less code
- Filtering data easily
- Sorting data
- Grouping data
- Joining data
- Selecting specific data
- Aggregating data
- Working with collections
- Database querying with Entity Framework

---

3. LINQ Namespace

To use LINQ with collections, import:

```
using System.Linq;
```

Example:

```
using System;
using System.Linq;
using System.Collections.Generic;
```

---

4. LINQ Syntax

LINQ mainly has two types of syntax:

1. Method Syntax

```
var result = numbers.Where(x => x > 20);
```

2. Query Syntax

```
var result = from number in numbers
             where number > 20
             select number;
```

Both produce similar results.

Method syntax is commonly used in modern C# applications.

---

5. Lambda Expression

LINQ commonly uses **lambda expressions**.

Example:

```
x => x > 10
```

Here:

- `x` → input
- `=>` → lambda operator
- `x > 10` → condition

Example:

```
var result = numbers.Where(x => x > 10);
```

---

6. Where() – Filtering Data

`Where()` is used to filter data based on a condition.

```
List<int> numbers = new List<int>
{
    10, 20, 30, 40, 50
};

var result = numbers.Where(x => x > 25);
```

Output:

```
30
40
50
```

Example with Employees

```
var employees = new List<Employee>
{
    new Employee { Id = 1, Name = "John", Salary = 25000 },
    new Employee { Id = 2, Name = "David", Salary = 40000 },
    new Employee { Id = 3, Name = "Mike", Salary = 30000 }
};

var result = employees
    .Where(x => x.Salary > 30000)
    .ToList();
```

This returns employees whose salary is greater than 30,000.

---

7. Select() – Selecting Data

`Select()` is used to select specific properties or transform data.

```
var names = employees
    .Select(x => x.Name)
    .ToList();
```

Output:

```
John
David
Mike
```

Selecting Multiple Properties

```
var result = employees
    .Select(x => new
    {
        x.Name,
        x.Salary
    })
    .ToList();
```

Output:

```
John   25000
David  40000
Mike   30000
```

---

8. OrderBy() – Sorting

`OrderBy()` sorts data in ascending order.

```
var result = employees
    .OrderBy(x => x.Salary)
    .ToList();
```

Result:

```
John
Mike
David
```

---

9. OrderByDescending()

Used to sort data in descending order.

```
var result = employees
    .OrderByDescending(x => x.Salary)
    .ToList();
```

Result:

```
David
Mike
John
```

---

10. ThenBy()

`ThenBy()` is used for secondary sorting.

```
var result = employees
    .OrderBy(x => x.Department)
    .ThenBy(x => x.Name)
    .ToList();
```

The data is first sorted by department and then by employee name.

---

11. First() and FirstOrDefault()

First()

Returns the first element.

```
var employee = employees.First();
```

If the collection is empty, `First()` throws an exception.

FirstOrDefault()

```
var employee = employees.FirstOrDefault();
```

If no element exists, it returns the default value (`null` for reference types).

Usually, `FirstOrDefault()` is safer when the collection may be empty.

---

12. Single() and SingleOrDefault()

Single()

Used when exactly one matching record is expected.

```
var employee = employees.Single(x => x.Id == 1);
```

`Single()` throws an exception if:

- No record exists
- More than one record exists

SingleOrDefault()

```
var employee = employees
    .SingleOrDefault(x => x.Id == 1);
```

Returns the matching record or default value if none exists.

---

13. Any() – Checking Existence

`Any()` checks whether at least one element satisfies a condition.

```
bool exists = employees.Any(x => x.Salary > 50000);
```

Returns:

```
true
```

or

```
false
```

Example

```
if (employees.Any(x => x.Name == "John"))
{
    Console.WriteLine("Employee exists");
}
```

---

14. All() – Checking All Records

`All()` checks whether every element satisfies a condition.

```
bool result = employees.All(x => x.Salary > 20000);
```

Returns `true` if every employee has a salary greater than 20,000.

---

15. Count()

`Count()` returns the number of elements.

```
int count = employees.Count();
```

With Condition

```
int count = employees.Count(x => x.Salary > 30000);
```

---

16. Sum()

`Sum()` calculates the total.

```
decimal totalSalary = employees.Sum(x => x.Salary);
```

Example:

```
25000 + 40000 + 30000 = 95000
```

---

17. Average()

`Average()` calculates the average value.

```
double averageSalary = employees.Average(x => x.Salary);
```

---

18. Min() and Max()

Minimum

```
var minimumSalary = employees.Min(x => x.Salary);
```

Maximum

```
var maximumSalary = employees.Max(x => x.Salary);
```

---

19. Take()

`Take()` returns a specified number of elements.

```
var result = employees.Take(2).ToList();
```

Returns the first two employees.

---

20. Skip()

`Skip()` skips a specified number of elements.

```
var result = employees.Skip(2).ToList();
```

Skips the first two employees.

---

21. Take() and Skip() – Pagination

LINQ can be used for pagination.

```
int pageNumber = 2;
int pageSize = 10;

var result = employees
    .Skip((pageNumber - 1) * pageSize)
    .Take(pageSize)
    .ToList();
```

For page 2:

```
Skip 10
Take 10
```

---

22. Distinct()

`Distinct()` removes duplicate values.

```
List<int> numbers = new List<int>
{
    10, 20, 20, 30, 30, 40
};

var result = numbers.Distinct().ToList();
```

Output:

```
10
20
```

```
30
40
```

---

23. Contains()

`Contains()` checks whether a collection contains a specific value.

```
bool result = numbers.Contains(20);
```

Returns:

```
true
```

---

24. Where() with Multiple Conditions

You can combine conditions using `&&` and `||`.

```
var result = employees
    .Where(x => x.Salary > 30000 && x.Department == "IT")
    .ToList();
```

Using OR:

```
var result = employees
    .Where(x => x.Department == "IT" ||
                x.Department == "HR")
    .ToList();
```

---

25. GroupBy()

`GroupBy()` groups data based on a property.

Example:

```
var result = employees
    .GroupBy(x => x.Department);
```

Display Groups

```
foreach (var group in result)
{
    Console.WriteLine(group.Key);

    foreach (var employee in group)
    {
        Console.WriteLine(employee.Name);
    }
}
```

Example output:

```
IT
John
David

HR
Mike
```

---

26. Join()

`Join()` is used to combine data from two collections based on a common key.

Example:

```
var result = employees.Join(
    departments,
    e => e.DepartmentId,
    d => d.Id,
    (e, d) => new
    {
        EmployeeName = e.Name,
        DepartmentName = d.Name
    });
```

This is similar to a SQL `INNER JOIN`.

---

27. SelectMany()

`SelectMany()` is used to flatten nested collections.

Example:

```
var students = new List<Student>
{
    new Student
    {
        Name = "John",
        Subjects = new List<string> { "C#", "SQL" }
    },
    new Student
    {
        Name = "David",
        Subjects = new List<string> { "Java", "HTML" }
    }
};

var subjects = students
    .SelectMany(x => x.Subjects)
    .ToList();
```

Output:

```
C#
SQL
Java
HTML
```

---

28. ToList()

`ToList()` converts the LINQ result into a `List<T>`.

```
var result = employees
    .Where(x => x.Salary > 30000)
    .ToList();
```

---

29. ToArray()

Converts the result into an array.

```
var result = employees
    .Where(x => x.Salary > 30000)
    .ToArray();
```

---

30. ToDictionary()

Converts data into a dictionary.

```
var result = employees
    .ToDictionary(x => x.Id, x => x.Name);
```

Example:

```
1 → John
2 → David
3 → Mike
```

---

31. Query Syntax

LINQ can also be written using SQL-like syntax.

```
var result = from employee in employees
             where employee.Salary > 30000
             select employee;
```

Multiple conditions:

```
var result = from employee in employees
             where employee.Salary > 30000
             orderby employee.Name
             select employee;
```

---

32. Method Syntax vs Query Syntax

Method Syntax

```
var result = employees
    .Where(x => x.Salary > 30000)
    .OrderBy(x => x.Name)
    .ToList();
```

Query Syntax

```
var result = from employee in employees
             where employee.Salary > 30000
             orderby employee.Name
             select employee;
```

Both are valid LINQ approaches.

---

33. Deferred Execution

Many LINQ queries are executed only when the results are actually enumerated.

Example:

```
var result = employees.Where(x => x.Salary > 30000);
```

The query may not execute immediately.

When you do:

```
var list = result.ToList();
```

the query is executed and the results are materialized into a list.

This behavior is called **Deferred Execution**.

---

34. Immediate Execution

Methods such as:

```
ToList()
ToArray()
Count()
Sum()
Average()
First()
```

cause the query to execute immediately.

Example:

```
var result = employees
    .Where(x => x.Salary > 30000)
    .ToList();
```

---

35. LINQ with Entity Framework

LINQ is commonly used with **Entity Framework Core** to query databases.

Example:

```
var employees = dbContext.Employees
    .Where(x => x.IsActive)
    .OrderBy(x => x.Name)
    .ToList();
```

The LINQ query is translated by Entity Framework into SQL and executed against the database.

Example conceptually:

```sql
SELECT *
FROM Employees
WHERE IsActive = 1
ORDER BY Name;
```

---

36. Real-World Example

Suppose we have:

```
public class Employee
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Salary { get; set; }
    public string Department { get; set; }
}
```

Data:

```
var employees = new List<Employee>
{
    new Employee
    {
        Id = 1,
        Name = "John",
        Salary = 25000,
        Department = "IT"
    },
    new Employee
    {
        Id = 2,
        Name = "David",
        Salary = 45000,
        Department = "IT"
    },
    new Employee
    {
        Id = 3,
        Name = "Mike",
        Salary = 35000,
        Department = "HR"
```

```
    }
};
```

Get IT Employees

```
var result = employees
    .Where(x => x.Department == "IT")
    .ToList();
```

Get Employee Names

```
var names = employees
    .Select(x => x.Name)
    .ToList();
```

Get Employees with Salary > 30,000

```
var result = employees
    .Where(x => x.Salary > 30000)
    .ToList();
```

Sort by Salary

```
var result = employees
    .OrderByDescending(x => x.Salary)
    .ToList();
```

Get Highest Salary

```
var highestSalary = employees
    .Max(x => x.Salary);
```

Check Whether an IT Employee Exists

```
bool exists = employees
    .Any(x => x.Department == "IT");
```

---

37. Common LINQ Methods

Method	Purpose
`Where()`	Filter data
`Select()`	Select/transform data
`OrderBy()`	Ascending sort
`OrderByDescending()`	Descending sort
`ThenBy()`	Secondary ascending sort
`ThenByDescending()`	Secondary descending sort
`First()`	Get first element
`FirstOrDefault()`	Get first or default
`Single()`	Get exactly one element
`SingleOrDefault()`	Get one or default
`Any()`	Check if any element exists
`All()`	Check if all elements satisfy condition
`Count()`	Count elements
`Sum()`	Calculate total
`Average()`	Calculate average
`Min()`	Find minimum
`Max()`	Find maximum
`Take()`	Take specified number
`Skip()`	Skip specified number
`Distinct()`	Remove duplicates
`Contains()`	Check for a value
`GroupBy()`	Group data
`Join()`	Join collections
`SelectMany()`	Flatten nested collections
`ToList()`	Convert to List
`ToArray()`	Convert to Array

Method	Purpose
`ToDictionary()`	Convert to Dictionary

---

38. Important LINQ Concepts to Learn

For a .NET developer, focus on these concepts:

Beginner

1. What is LINQ?
2. `Where()`
3. `Select()`
4. `OrderBy()`
5. `OrderByDescending()`
6. `First()`
7. `FirstOrDefault()`
8. `Any()`
9. `Count()`
10. `ToList()`

Intermediate

1. `GroupBy()`
2. `Join()`
3. `SelectMany()`
4. `Distinct()`
5. `Skip()`
6. `Take()`
7. `Single()`
8. `SingleOrDefault()`
9. `Sum()`
10. `Average()`
11. Deferred Execution
12. Immediate Execution

Advanced

1. LINQ with Entity Framework Core
2. `IEnumerable<T>`
3. `IQueryable<T>`
4. Expression Trees
5. LINQ-to-Objects
6. LINQ-to-Entities
7. Query optimization

8. Performance considerations

---

39. Practice Questions

Basic

1. Find all numbers greater than 50 using LINQ.
2. Find all even numbers.
3. Find all odd numbers.
4. Find the maximum number.
5. Find the minimum number.
6. Find the total of all numbers.
7. Find the average.
8. Sort numbers ascending.
9. Sort numbers descending.
10. Remove duplicate numbers.

Employee-Based

1. Get all employees from the IT department.
2. Get employees whose salary is greater than 30,000.
3. Get only employee names.
4. Find the highest salary.
5. Find the lowest salary.
6. Count employees.
7. Check whether an employee exists.
8. Group employees by department.
9. Sort employees by salary.
10. Get the top 3 highest-paid employees.

---

40. Quick Example

```
var result = employees
    .Where(x => x.Salary > 30000)
    .OrderByDescending(x => x.Salary)
    .Select(x => new
    {
        x.Name,
        x.Salary
    })
    .Take(3)
    .ToList();
```

This single LINQ query:

1. Filters employees with salary greater than 30,000.
2. Sorts them by salary from highest to lowest.
3. Selects only `Name` and `Salary`.
4. Takes the top 3 employees.
5. Converts the result to a `List`.

Conclusion

LINQ is one of the most important features for a C#/.NET developer.

The most commonly used LINQ methods are:

```
Where()
Select()
OrderBy()
OrderByDescending()
FirstOrDefault()
Any()
Count()
GroupBy()
Join()
Take()
Skip()
ToList()
```

Understanding these methods provides a strong foundation for working with collections and **Entity Framework Core database queries**.